

נ.נ. בתכנון משולב חומרה/תוכנה

אביב – 00460882

תרגול 4 – זיכרון וירטואלי

אוהד איתן

Efficient Memory Management for Large Language Model Serving with PagedAttention

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. arXiv:2309.06180.

Agenda

Problem

Background

Solution

Evaluation

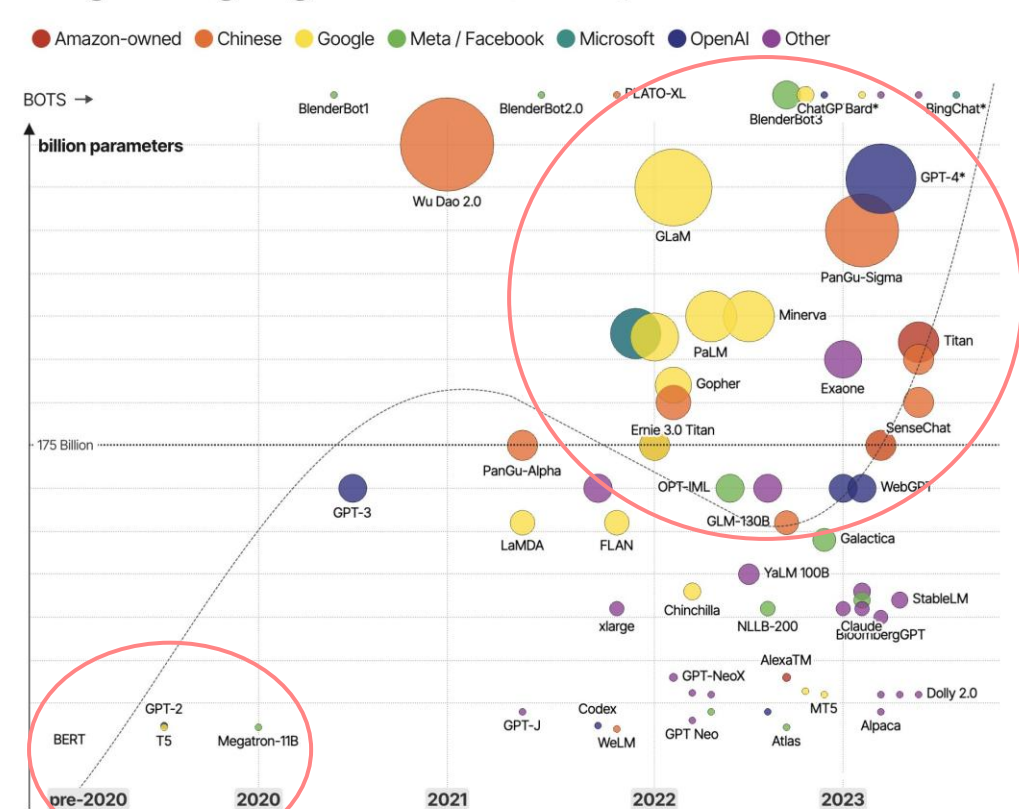
Summary

The Evolution of Large Language Models

Each increase in model size means billions more parameters to store

Modern LLMs require hundreds of GBs of GPU memory

The Rise and Rise of A.I. Large Language Models (LLMs) & their associated bots like ChatGPT

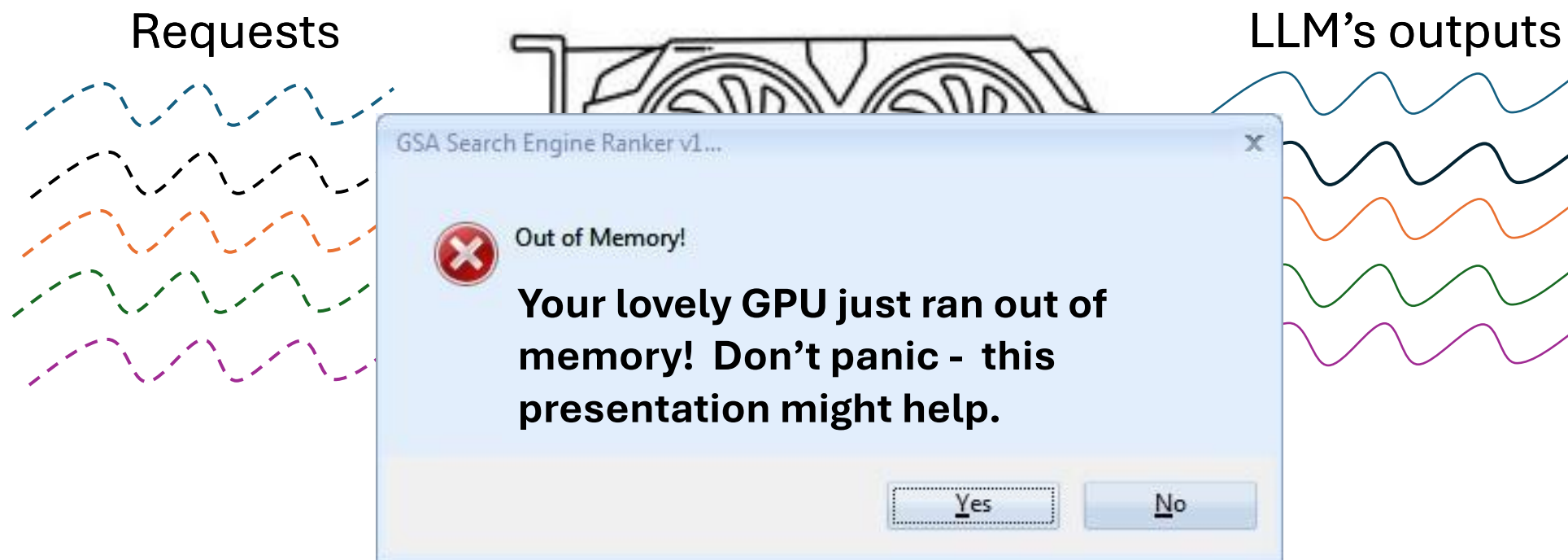


David McCandless, Tom Evans, Paul Barton
Information is Beautiful // May 2023

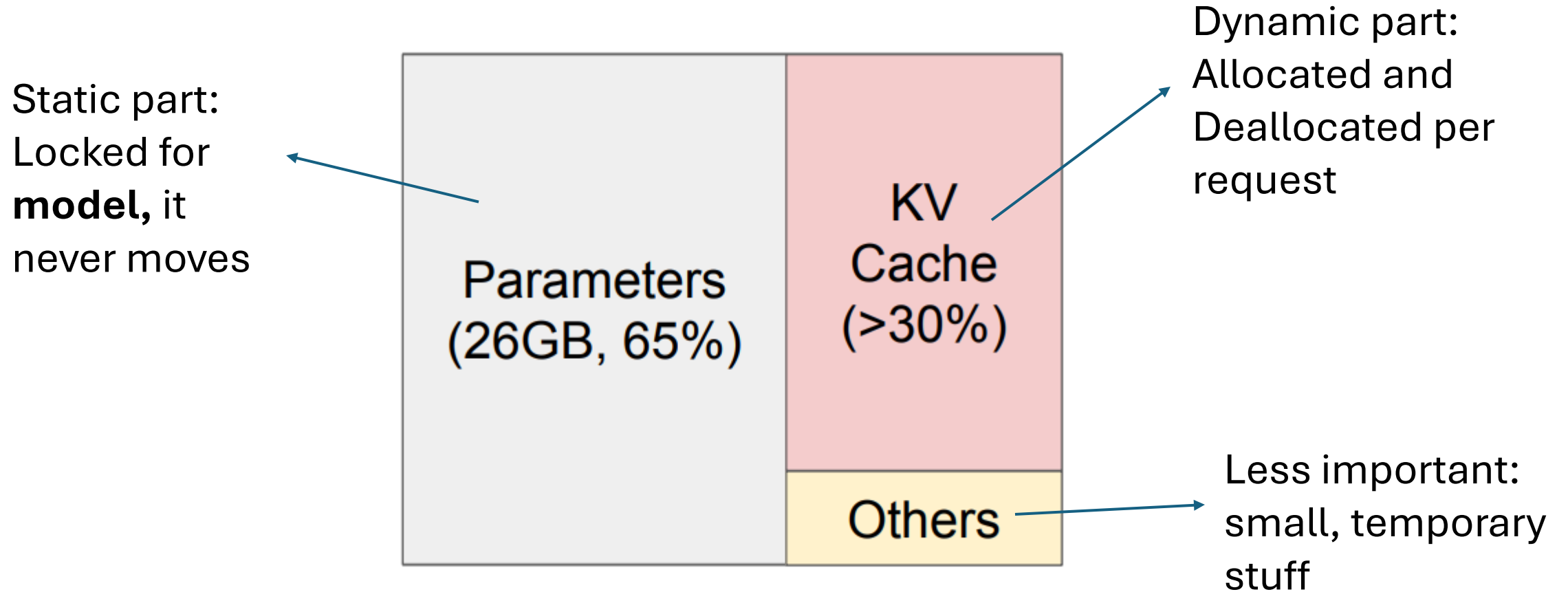
source: news reports, [LifeArchitecture.ai](#)
* = parameters undisclosed // see [the data](#)

MADE WITH [VIZsweat](#)

Today's Solution

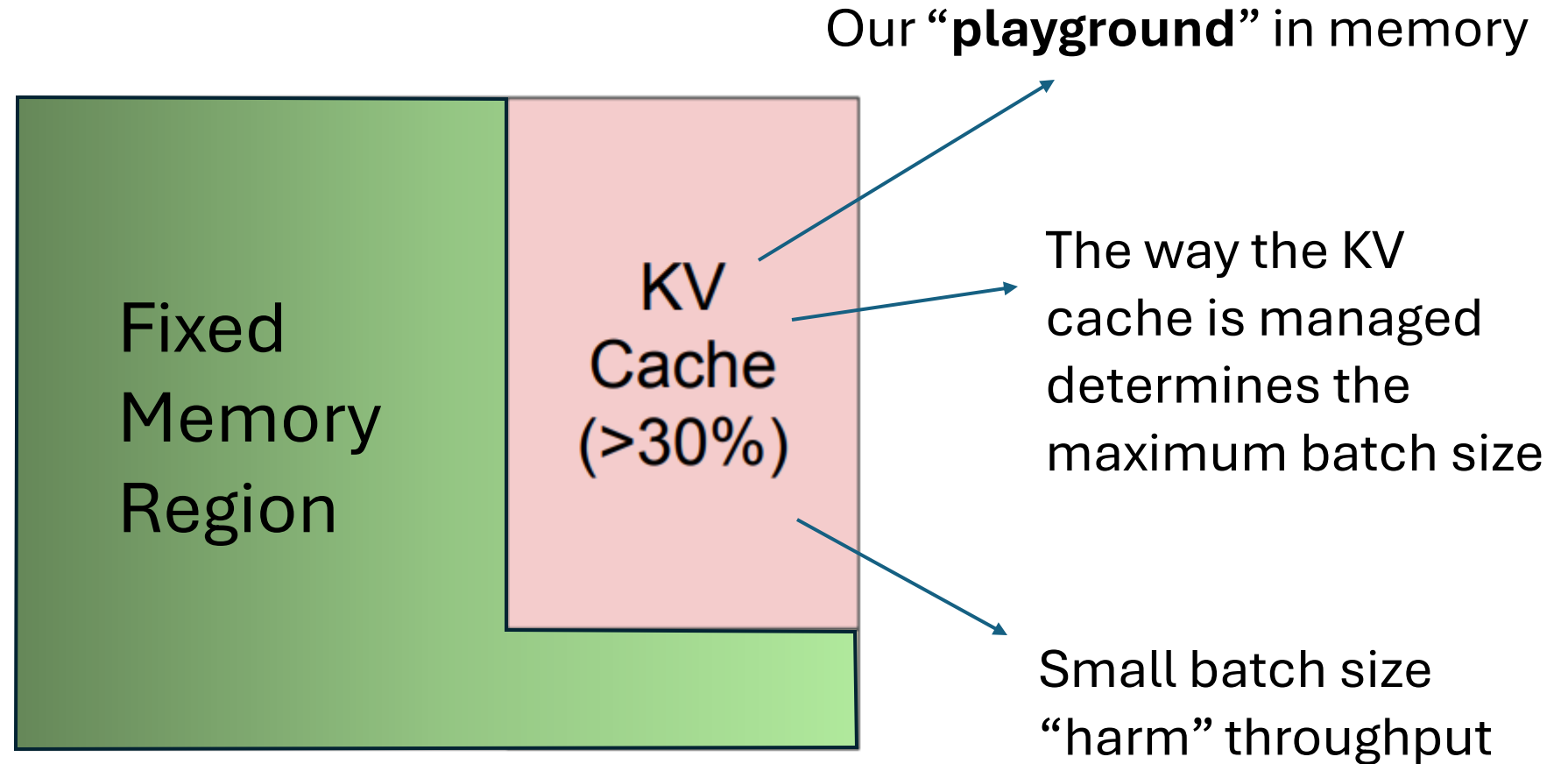


Where does the GPU Memory Go?

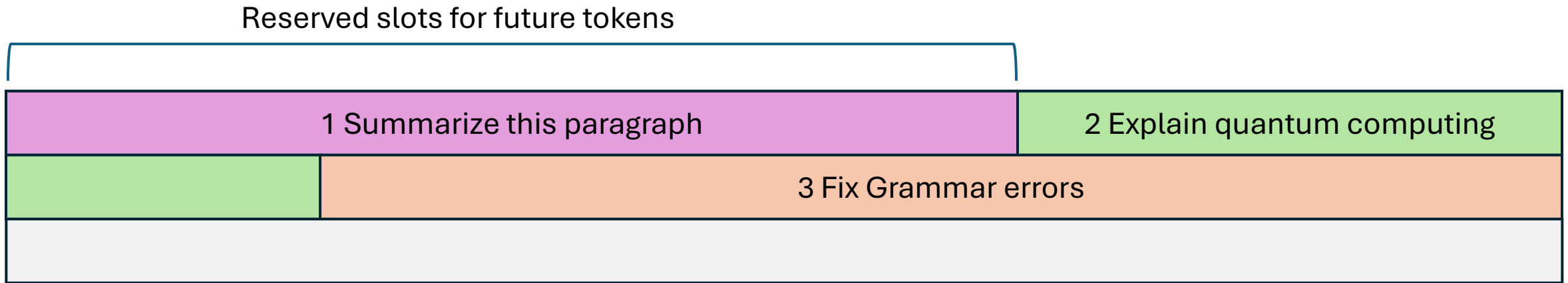


13B-parameter LLM on an NVIDIA A100 GPU with 40GB RAM

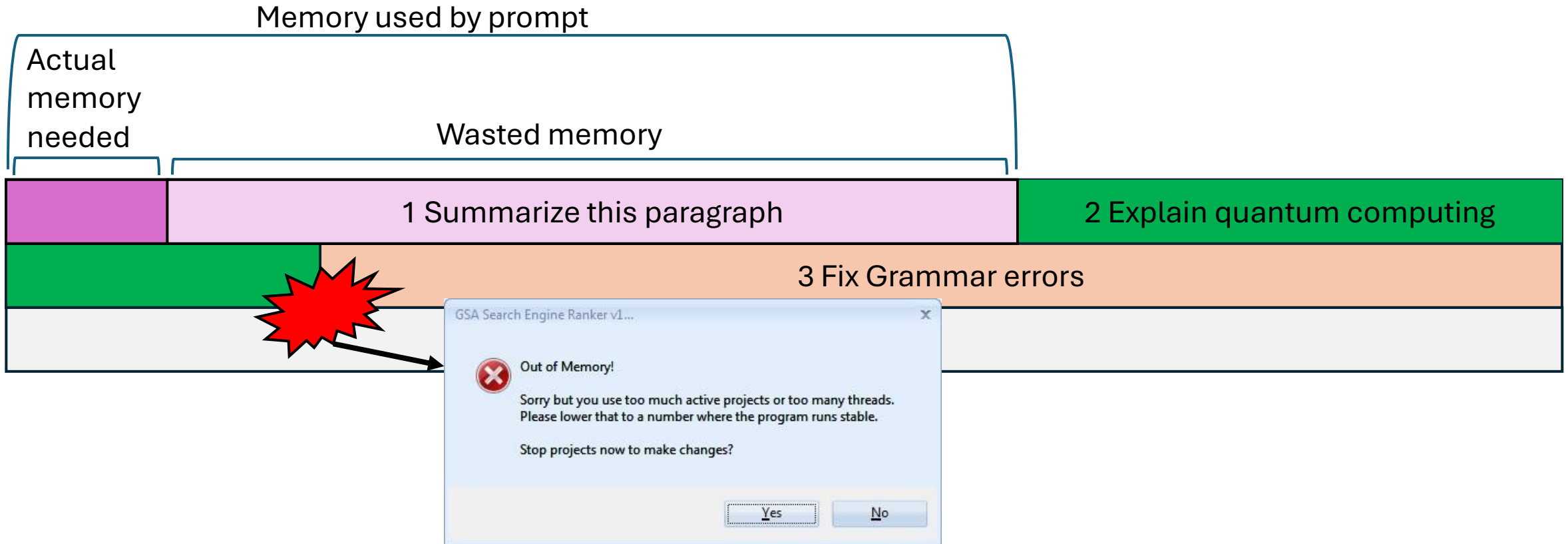
Where the GPU Memory Goes



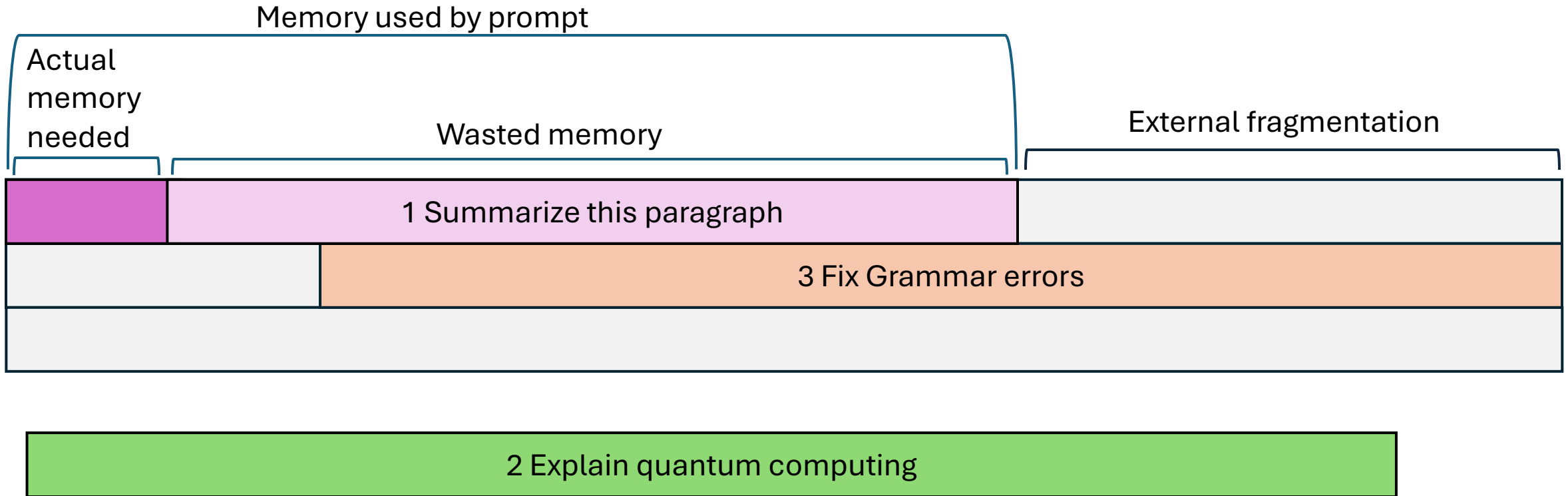
How Existing Systems Operate



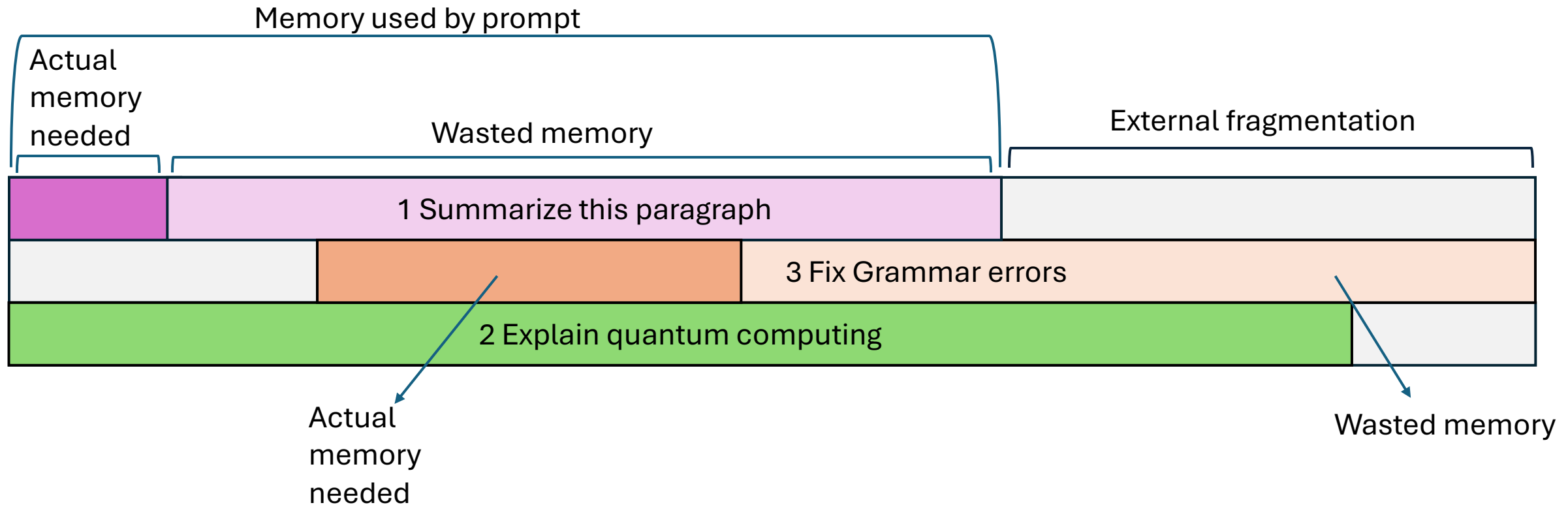
How Existing Systems Operate



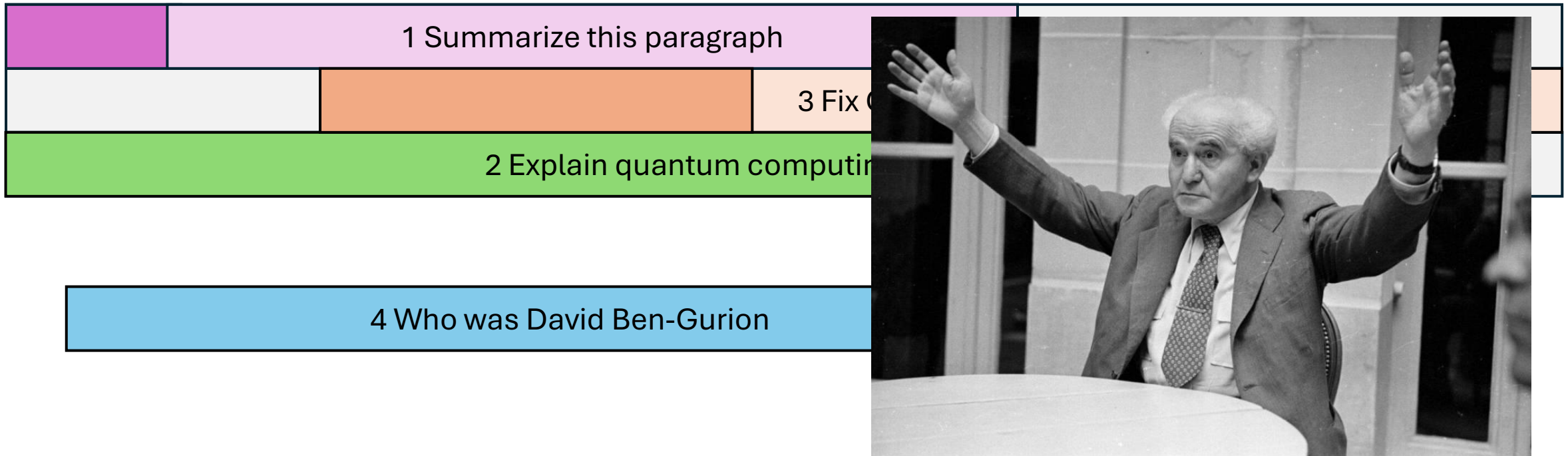
How Existing Systems Operate



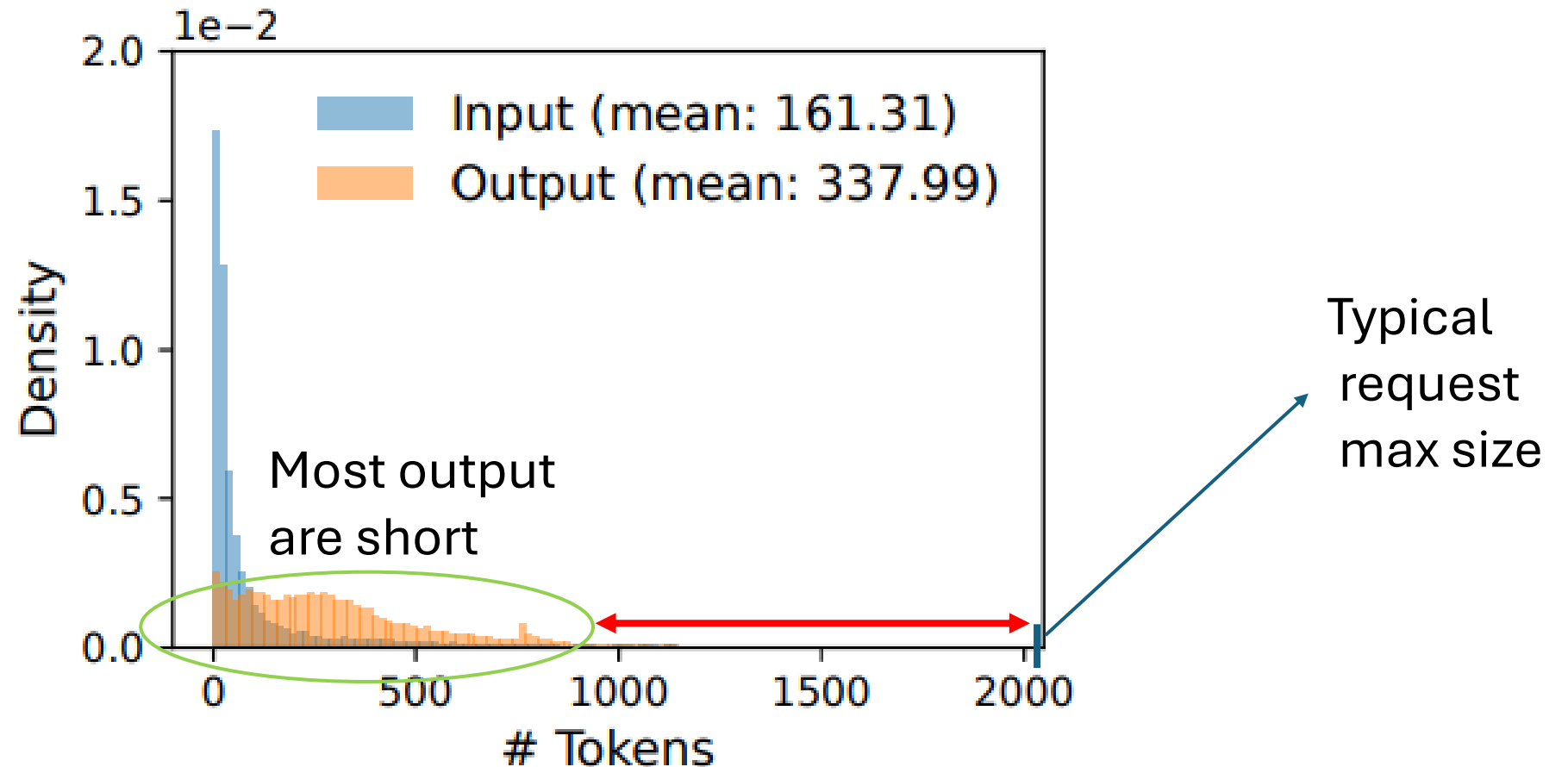
How Existing Systems Operate



How Existing Systems Operate



Existing Approach - Inefficient Memory Use



Attention

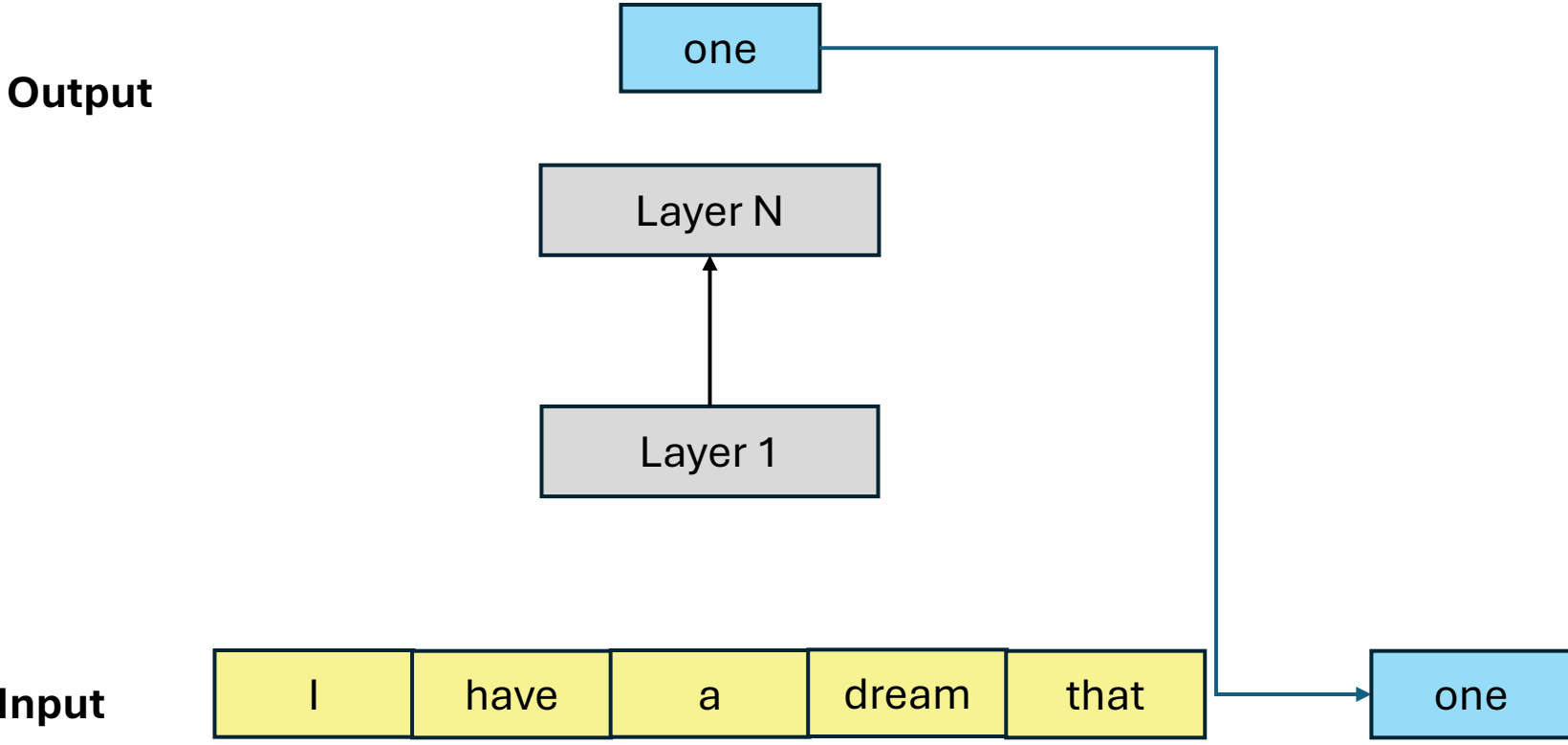
A way for a model to “look back” at previous words when generating the next one.

Query (Q) – “What am I looking for?”

Key (K) – “What do I contain?”

Value (V) – “What information should I pass on?”

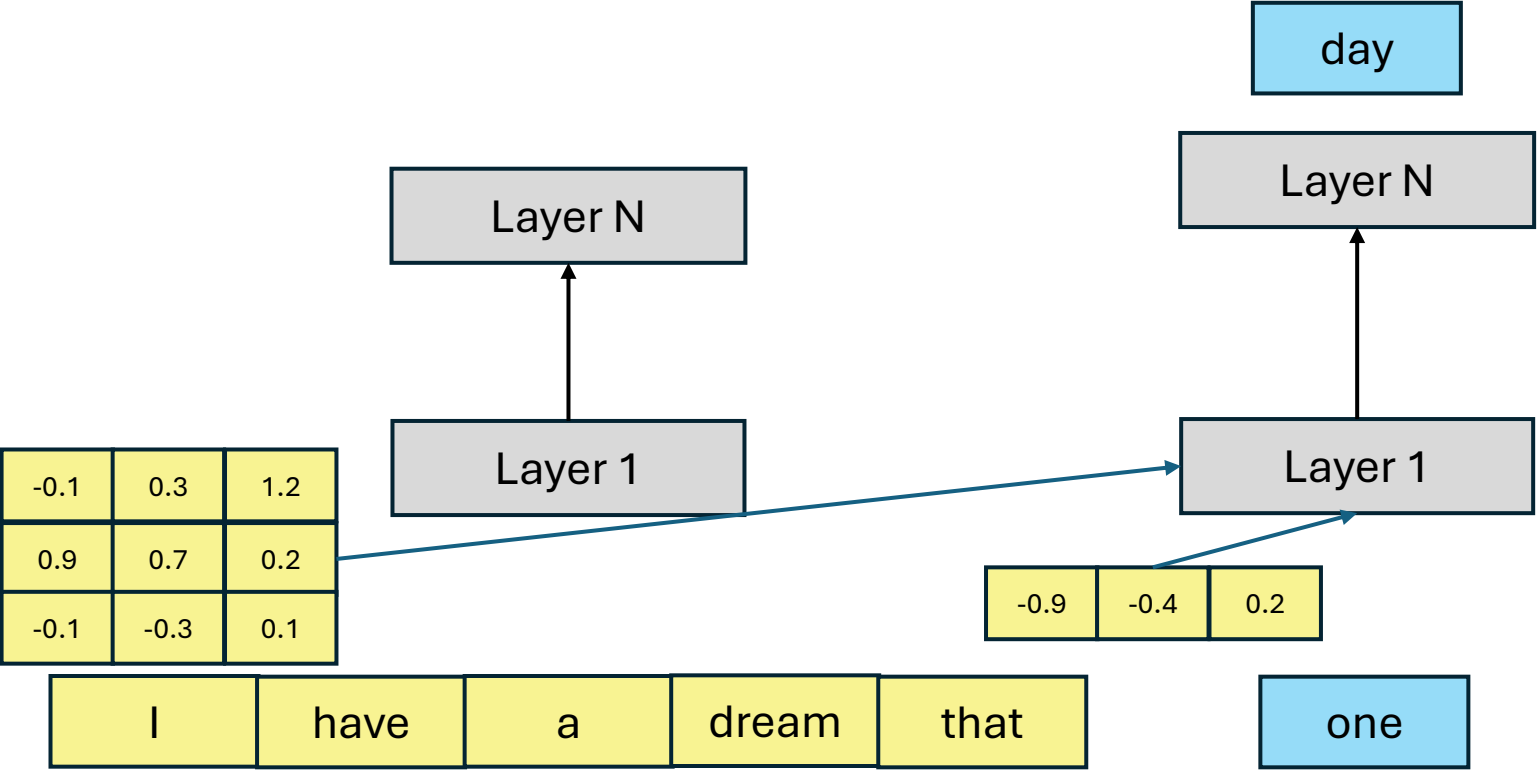
Attention



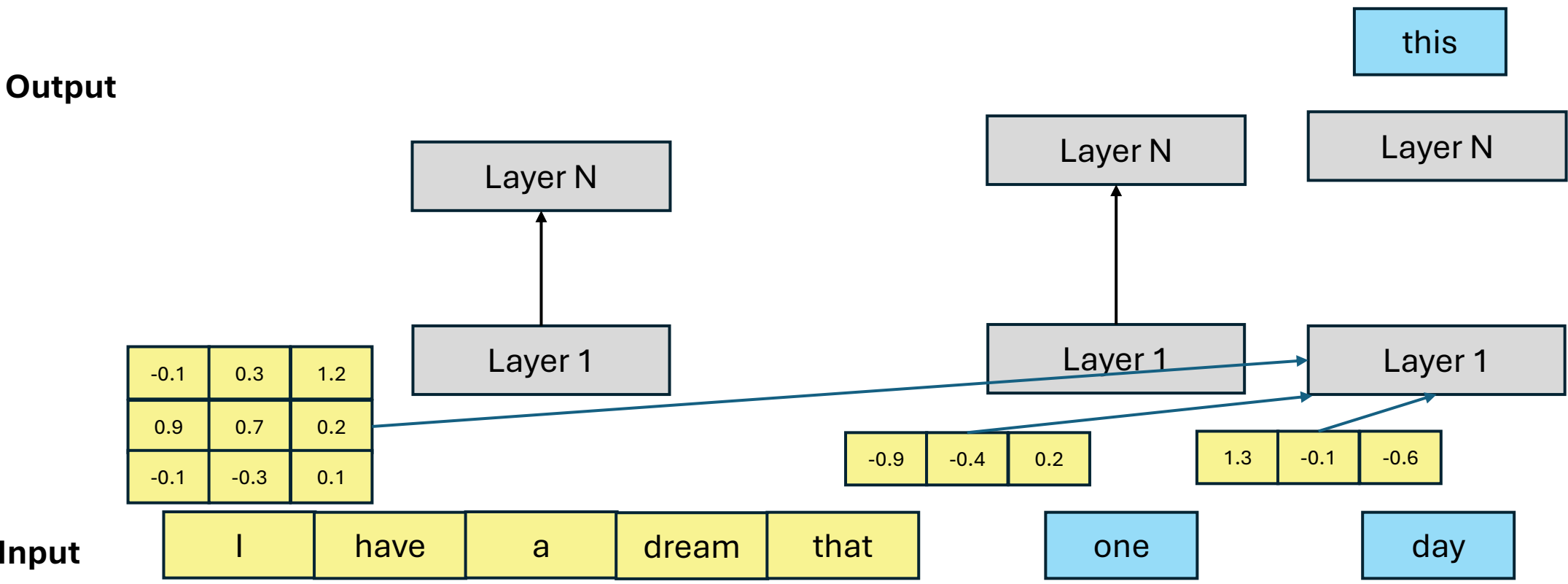
Attention

Output

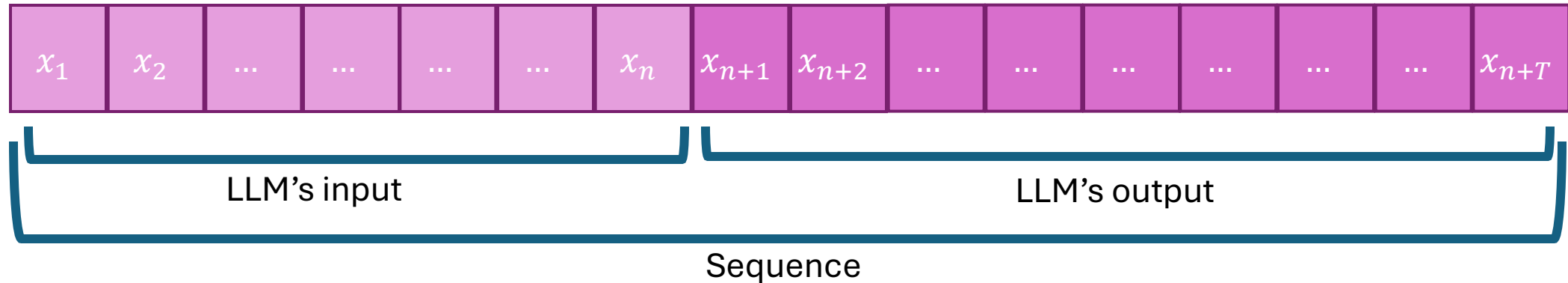
Input



Attention



Autoregressive Generation Phase



Each token depending on all previous ones

Key and value vectors are cached for future tokens

A token's KV cache varies by its position in the sequence

The KV Cache Problem

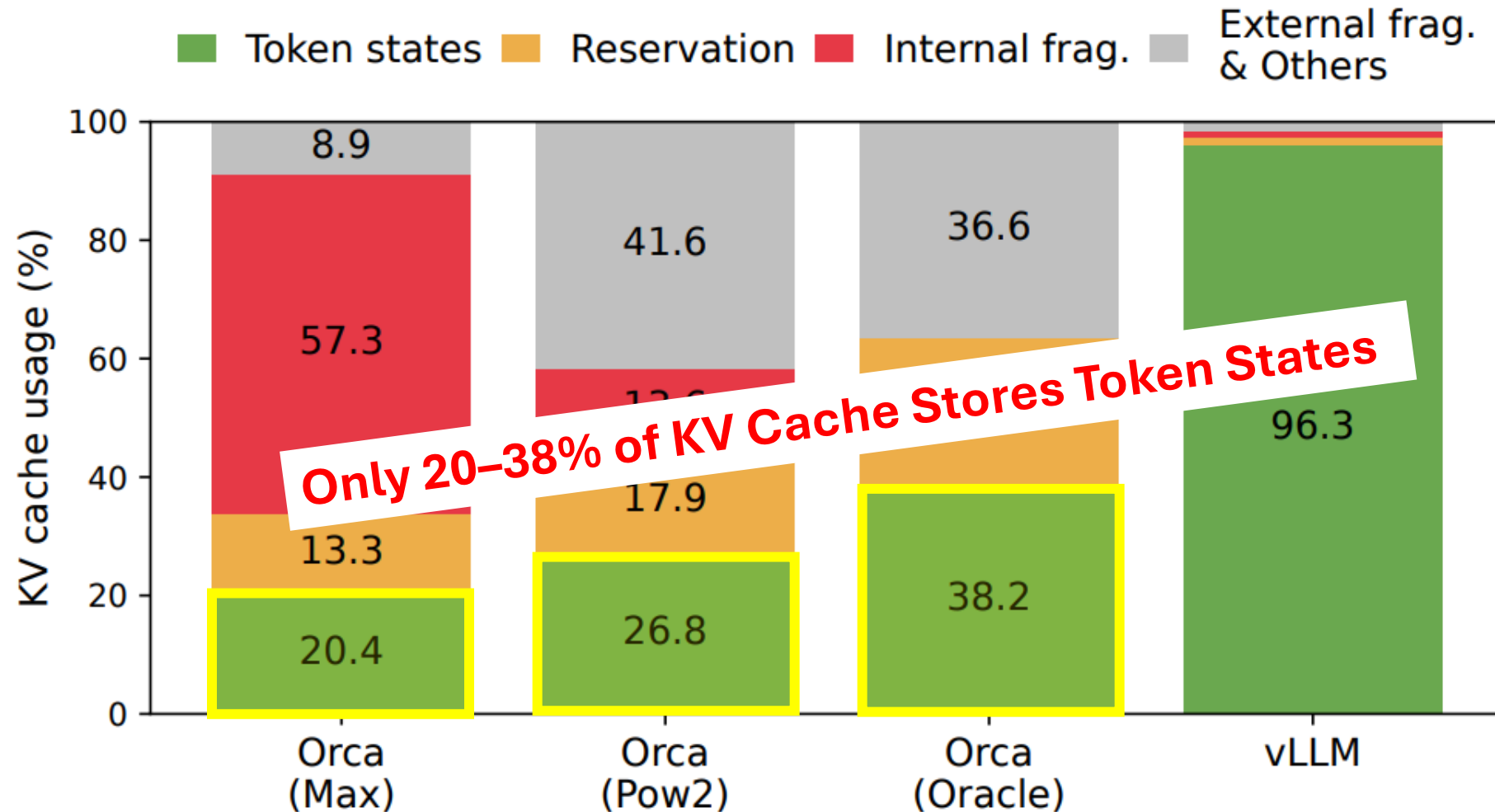
For OPT-13B, a single token needs ≈ 800 KB of GPU memory

$$2 \text{ (key, vector)} \times 5120 \text{ (hidden)} \times 40 \text{ (\#layers)} \times 2B \text{ (FP16)}$$

Up to 2048 tokens $\rightarrow \sim 1.6$ GB per request

Even with 14 GB KV cache (40 GB GPU), can host only a few dozen requests at once

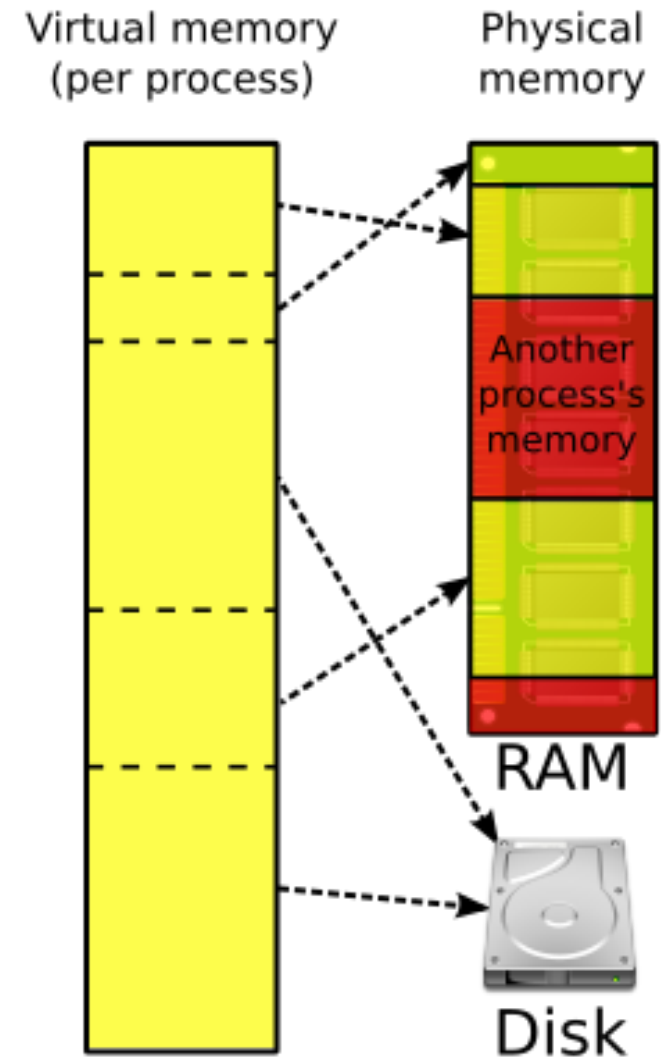
Current System Are inefficient



Bringing OS Concepts to LLMs

Our system adapts the idea of virtual memory for LLMs

Implements common virtual memory routines and mechanisms in software

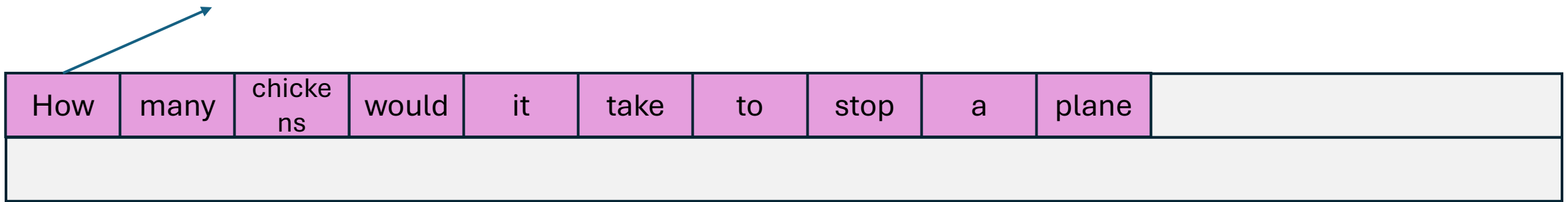


The Solution - PagedAttention

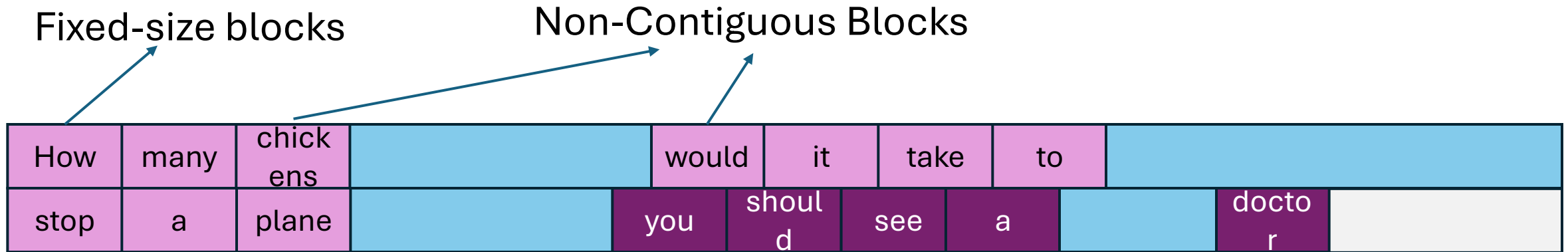
How many chickens would it take to stop a plane?

The Solution - PagedAttention

Fixed-size blocks



The Solution - PagedAttention



Blocks as pages

Memory sharing like libraries

Requests as processes

Block allocating on demand as lazy malloc

Alleviates internal fragmentation

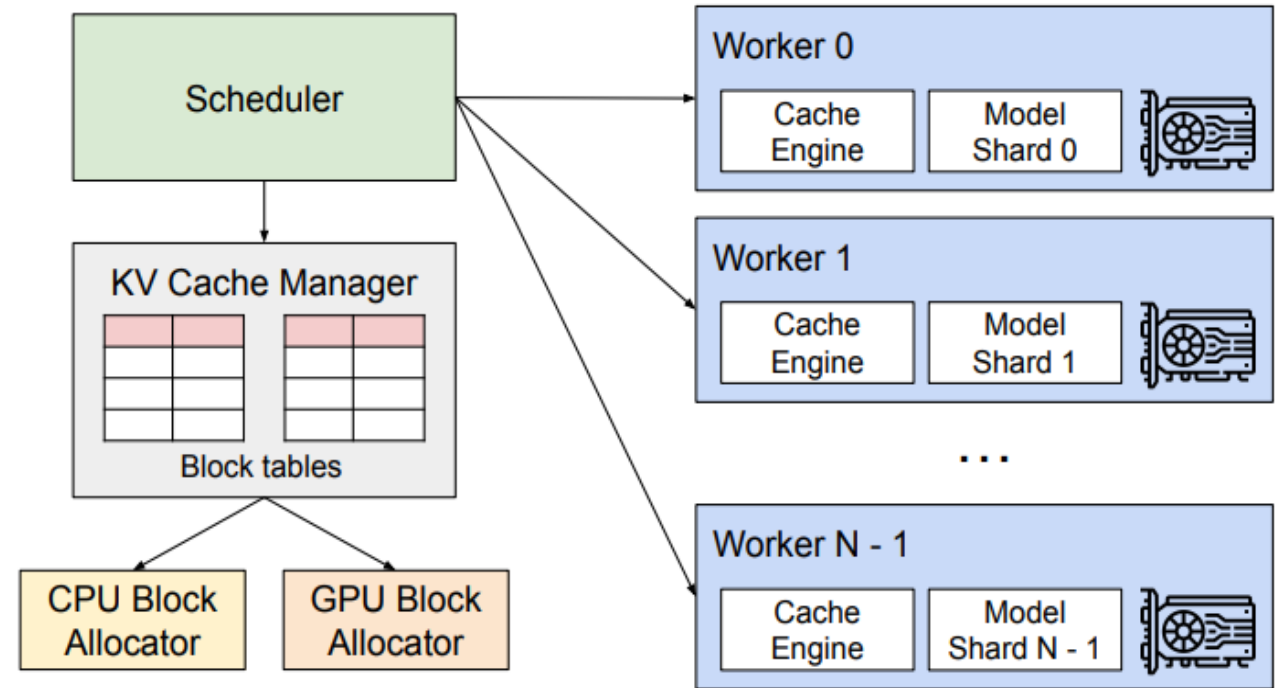
Eliminates external fragmentation

Tokens as bytes

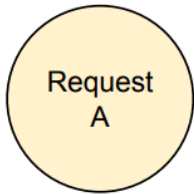
PagedAttention and the vLLM Architecture

Introduces **PagedAttention**, a new attention algorithm designed for **efficient KV cache management**

Implements **vLLM**, a high-throughput LLM serving engine



PagedAttention in Action



Request
A

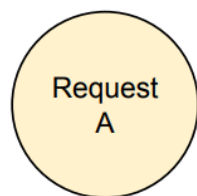
Prompt: *"Four score and seven years ago our"*

Outputs: *"fathers" → "brought" → ...*

Logical KV blocks

Block 0	① <i>Four</i>	① <i>score</i>	① <i>and</i>	① <i>seven</i>
Block 1	① <i>years</i>	① <i>ago</i>	① <i>our</i>	② <i>fathers</i>
Block 2	③ <i>brought</i>			
Block 3				

PagedAttention in Action



Prompt: "Four score and seven years ago our"
Outputs: "fathers" → "brought" → ...

Logical KV blocks

Block 0	① Four	① score	① and	① seven
Block 1	① years	① ago	① our	② fathers
Block 2	③ brought			
Block 3				

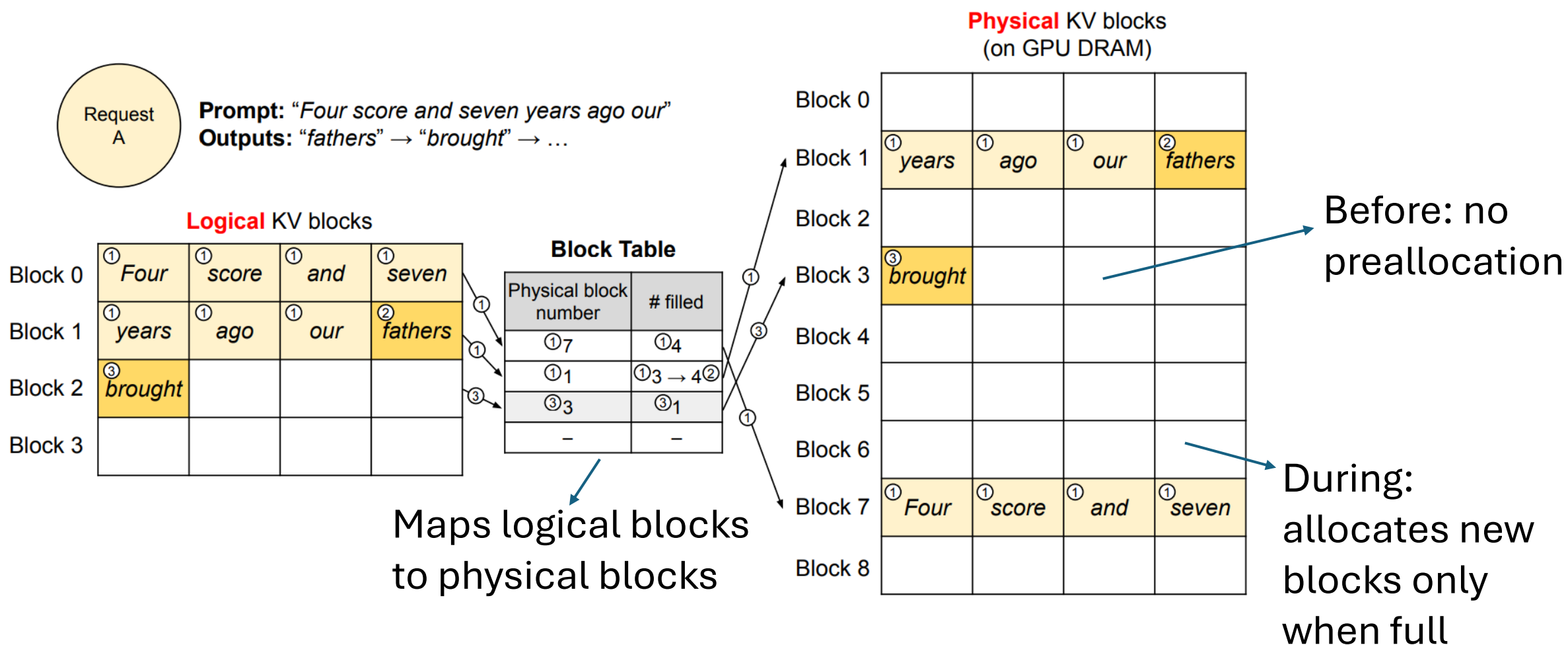
Physical KV blocks
(on GPU DRAM)

Block 0				
Block 1	① years	① ago	① our	② fathers
Block 2				
Block 3	③ brought			
Block 4				
Block 5				
Block 6				
Block 7	① Four	① score	① and	① seven
Block 8				

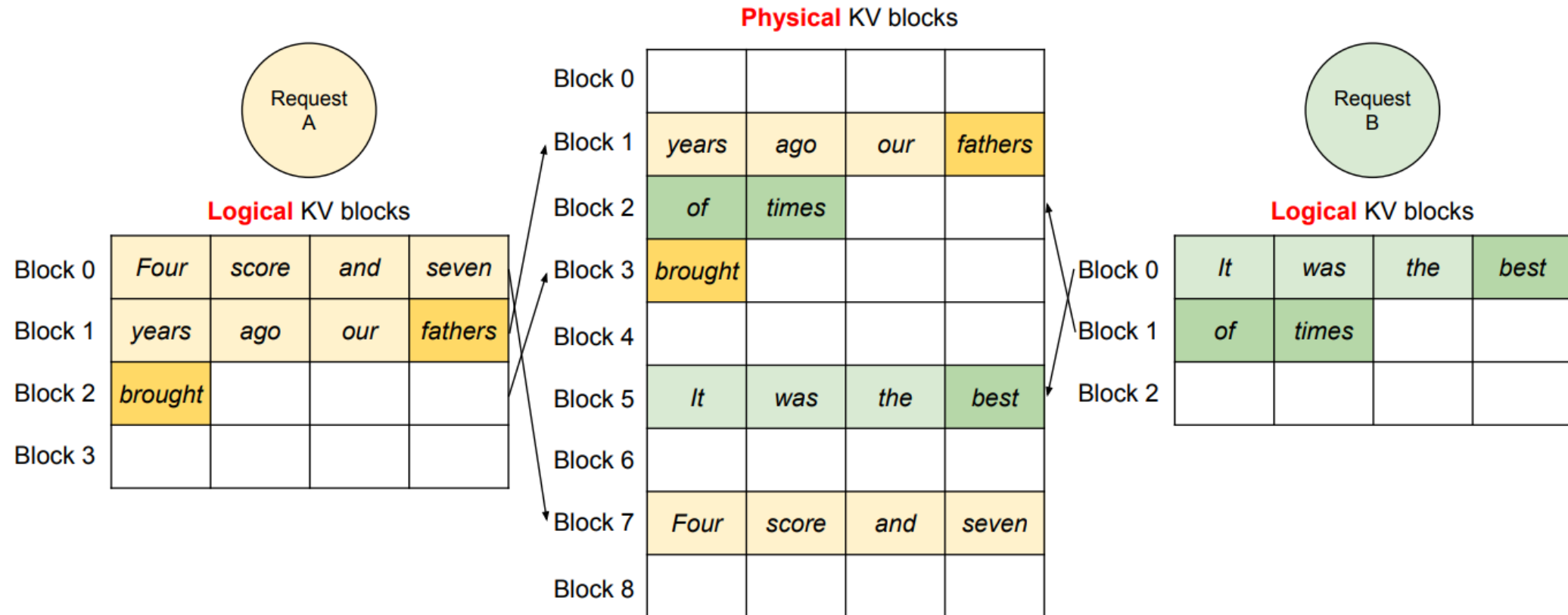
Before: no
preallocation

During:
allocates new
blocks only
when full

PagedAttention in Action



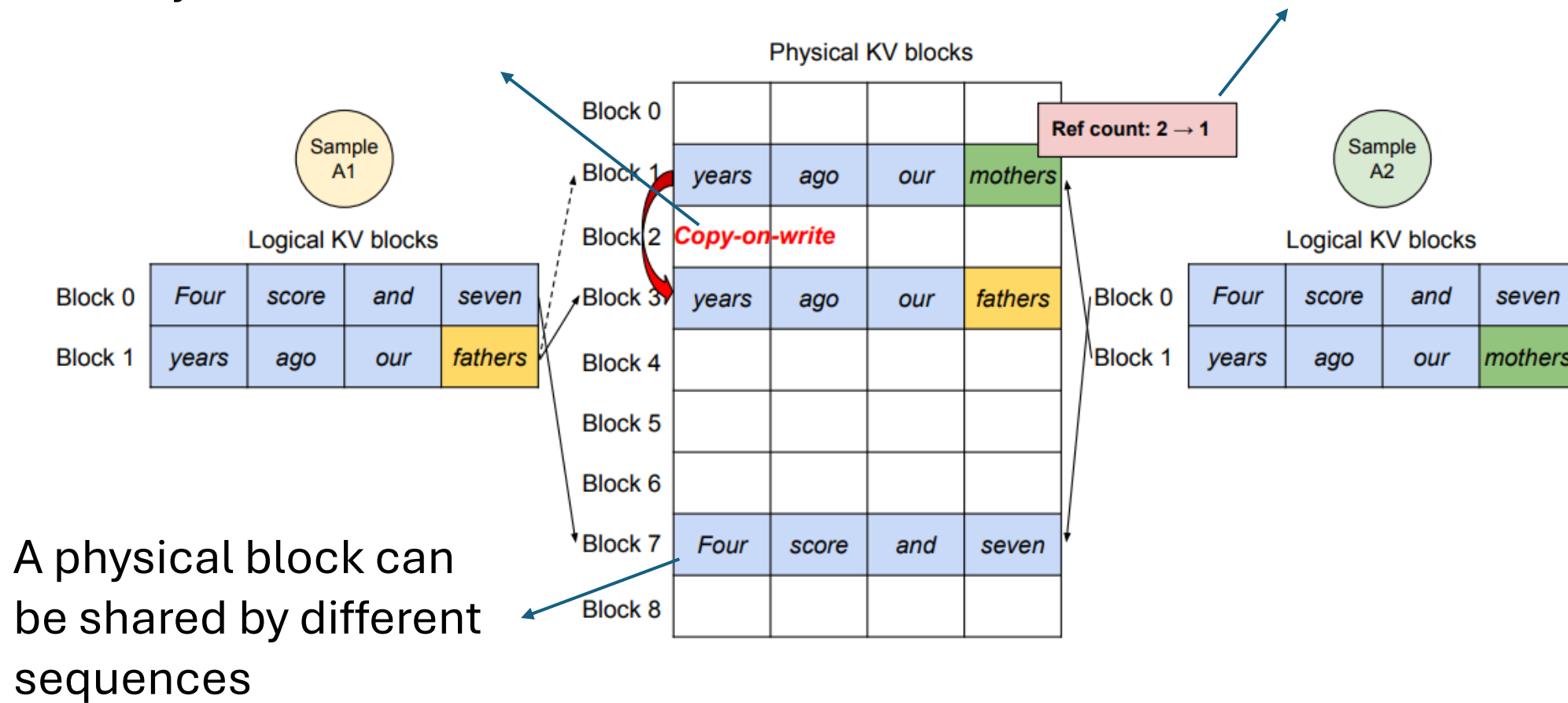
Parallel Decoding with Shared GPU Memory



Parallel Decoding Example

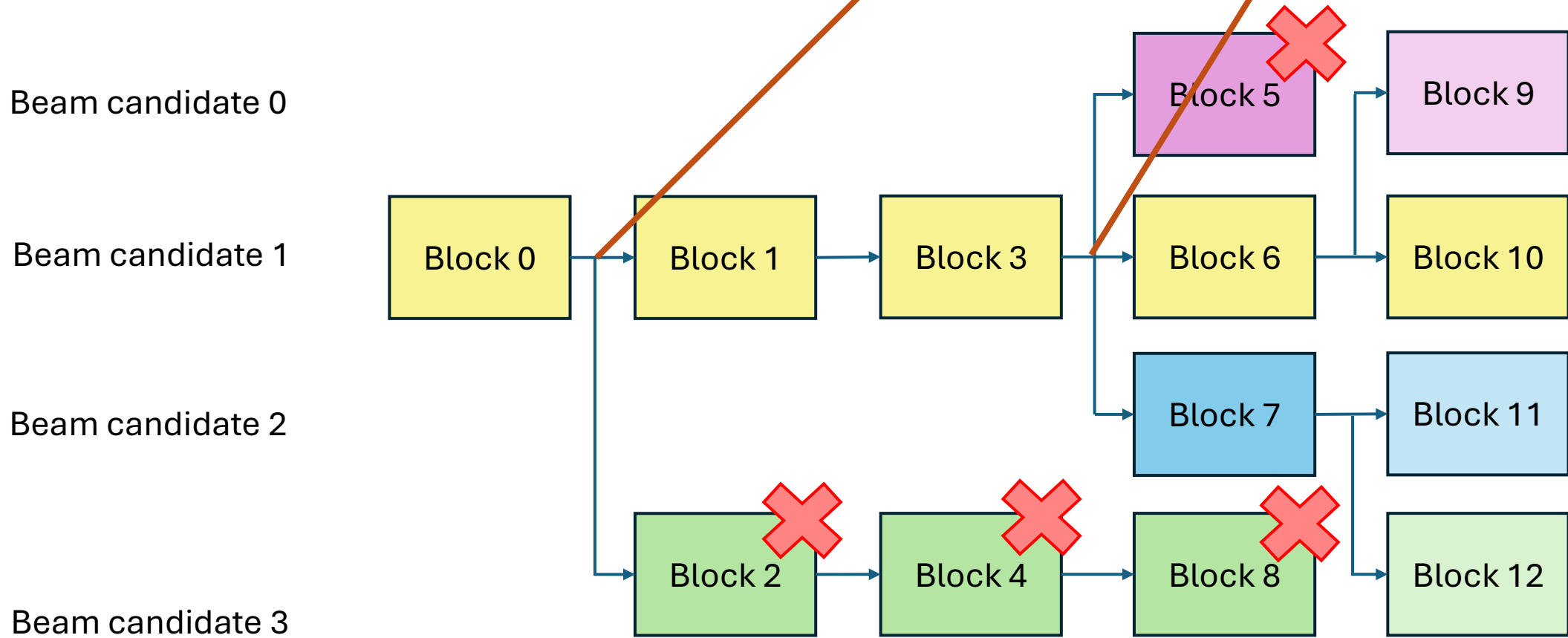
Duplicated only when modified

Each physical block has a **reference count** to track shared use



Beam Search Example

Copy-on-write occurs only when a shared block is modified



Shared Prefix

You are Gordon Ramsay (pls stay calm), write an emotional recipe for pizza.

Dough, sauce, cheese, olive oil, basil, bake, slice, serve. 🍕

Think on the shared prefix like libc

You are Gordon Ramsay (pls stay calm), write an emotional recipe for revenge.

Preheat rage to 400°, mix betrayal and garlic, bake until justice smells done. 🔥

Only the task-specific part recomputes

Evaluation - Orca

A memory management system for LLM inference efficiently allocate and reuse GPU memory, especially the KV cache

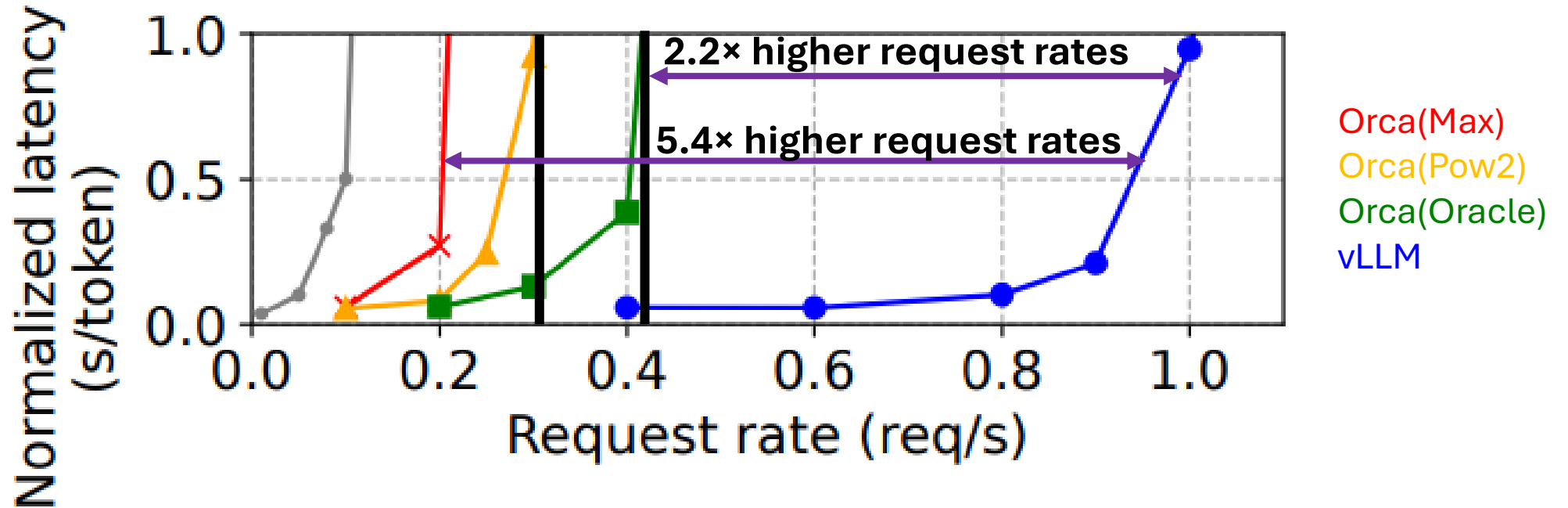
Orca (Oracle): assumes perfect knowledge of output lengths

Orca (Pow2): over-reserves output space by $\leq 2^x$

Orca (Max): reserves full max sequence length (2048 tokens)

System Behavior at Scale

↑ Request rate → ↑ Latency grows ... ↑ ↑ ↑ Latency grows

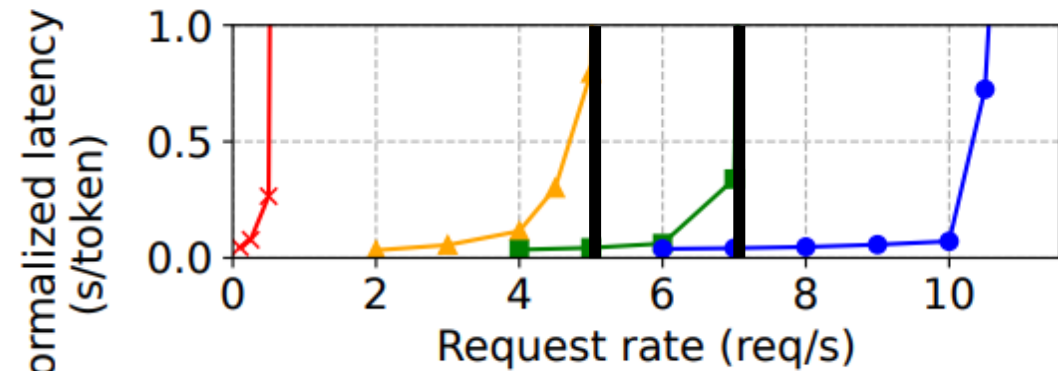


OPT-66B, 4 GPUs, ShareGPT

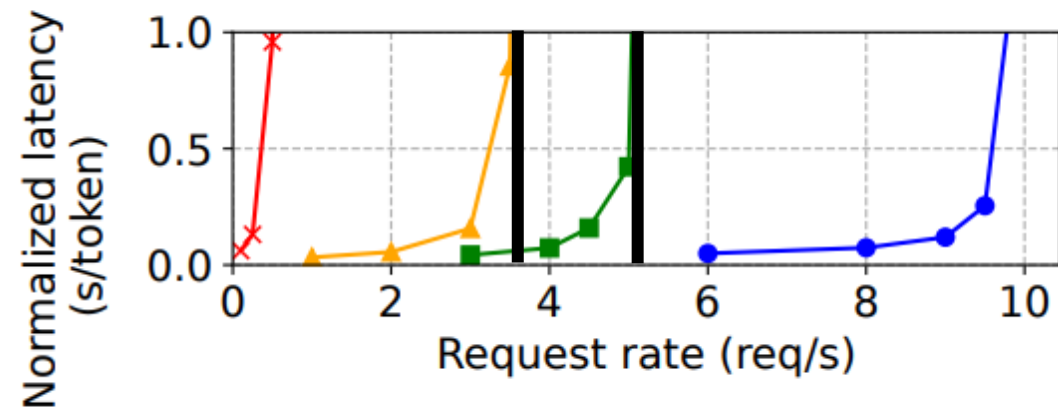
Memory Sharing Efficiency

Parallel Sampling → all sequences reuse the same KV cache

Beam Search → enables even greater KV cache sharing



parallel generation (parallel size = 4)



beam search (beam width = 4)

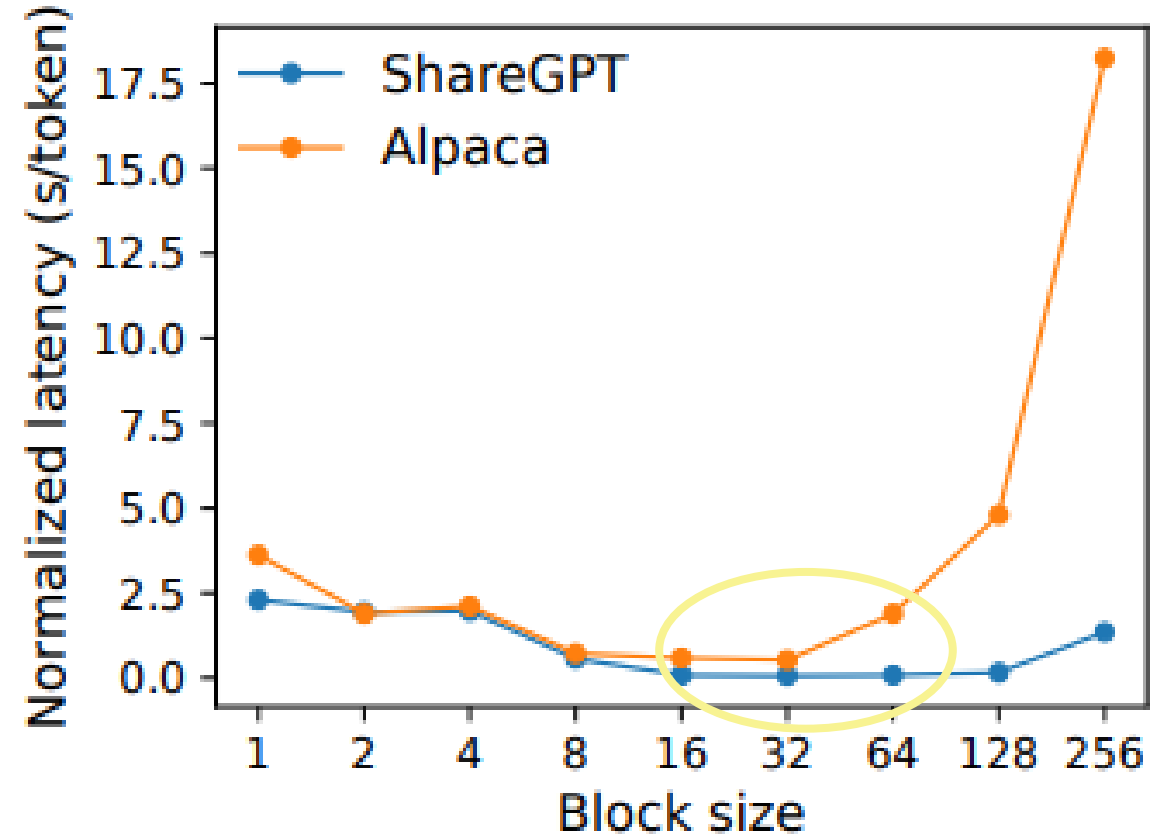
Orca(Max)
Orca(Pow2)
Orca(Oracle)
vLLM

Impact of Block Size

Block size strongly affects **vLLM performance**

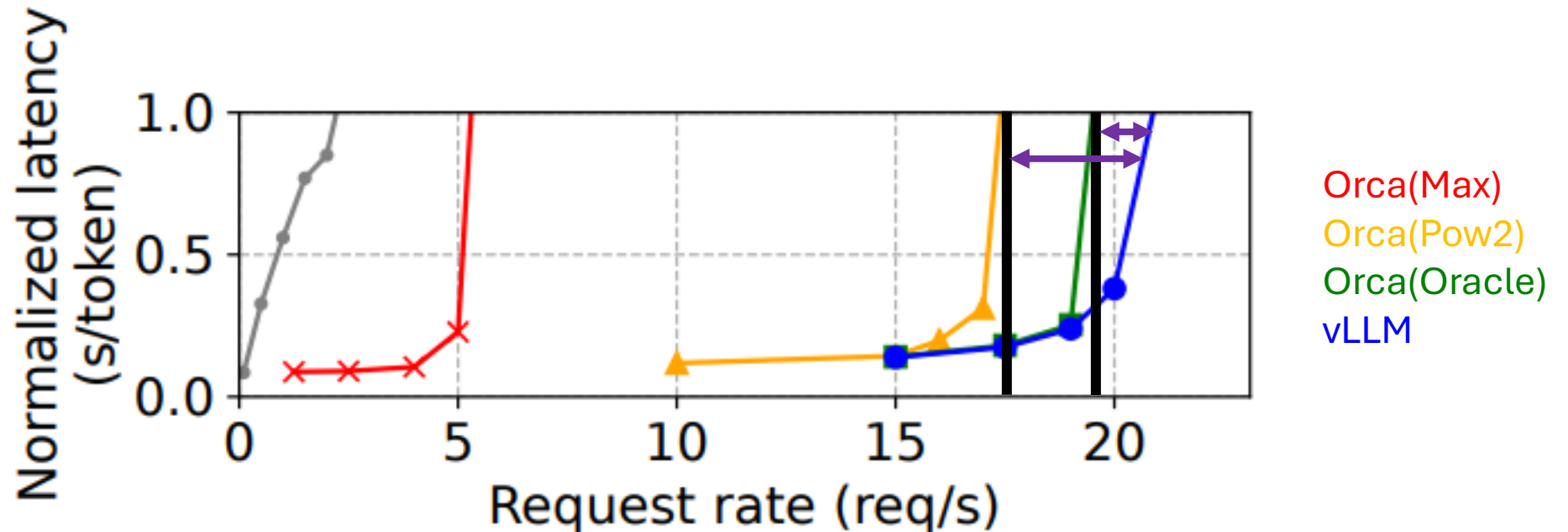
Too small → underutilized GPU parallelism

Too large → internal fragmentation and reduced sharing



Alpaca Dataset Results

In **OPT-175B** has **huge GPU memory**, and **Alpaca's** sequences are **short**, making the system **compute-bound**



OPT-175B, 8 GPUs, Alpaca

What We Covered Today

Identify the challenges in memory allocation

Propose PagedAttention, an attention algorithm that operates on KV cache, which is inspired by the virtual memory and paging in OS

Design and implement vLLM, a distributed LLM serving engine

Evaluate vLLM on various scenarios and demonstrate

Outperforms previous state-of-the-art solutions

Main Takeaways

Reduces memory fragmentation with paging

Reduces memory usage with KV block sharing

Enables batching of more requests, increasing the throughput of LLM inference

vLLM achieves 2-4× throughput improvements over the state-of-the-art systems

